R8 **Student's written exercises**
R9
R10 Module 2 assignment. NLP in Cloud Environment

Module 2 of the Cloud Computing course focuses on Natural Language Processing (NLP) in cloud environments, with a specific emphasis on sentiment analysis. The objective of this assignment is to train and evaluate a sentiment analysis model using a textual dataset and classical machine learning techniques, including Term Frequency-Inverse Document Frequency (TF-IDF) vectorization and a scikit-learn classification model such as Linear Support Vector Classifier (LinearSVC). The practical work is carried out in a cloud-based environment using Google Colab.

In this assignment, I followed the example "Training the sentiment analysis model example (colab)" given by the teacher (Pajula 2024). I used the "Sentiment Analysis Dataset" recommended by the teacher (Shrivastava 2021). The structure of the given example is as follows:

1. Importing Required Library, pandas.
2. Checking for Missing Values and removing them.
3. Preparing the Data for Training.
4. Creating and Training the Model.
5. Evaluating the Model.
6. Visualizing the Confusion Matrix.
7. Making Predictions with the Model.

This structure will be maintained throughout this report (Pajula 2024). Here is my completed practical work "Module2_Maksym.ipynb".

Importing Required Library, pandas

The necessary pandas, numpy and scikit-learn libraries for data manipulation and machine learning were imported:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report
```

Then I uploaded the CSV (Files-Upload-test.csv). To verify the data structure, I loaded the CSV file and inspected the first five rows:

```
df = pd.read_csv('test.csv', encoding='latin-1')
df.head()
```

The resulting DataFrame contained 9 columns, matching the original dataset's specifications.

Checking for Missing Values and removing them

I checked for null values using *df.isnull().sum()*, which identified 1 281 missing entries (NaN). To ensure data quality, I removed these rows using *df.dropna(inplace=True)*. A subsequent check confirmed that 0 missing values remained.

Then I performed a descriptive statistical analysis using *df.describe()*. This command generated an 8-parameter summary (count, mean, std, min, 25%, 50%, 75%, max). These statistics were only generated for three numerical columns: Population -2020, Land Area (Km^2), and Density (P/Km^2), as pandas automatically excludes categorical text columns from statistical aggregation.

Preparing the Data for Training

The "text" column was selected as input features (X), and the "sentiment" column was defined as the target variable (y). Used code:
df = df[['text', 'sentiment']]
df = df.dropna()// deletes all lines that contain at least one NaN

X = df['text']
y = df['sentiment']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

The dataset was split into training and testing subsets. The training data (X_train, y_train) was used to train the model, while the testing data (X_test, y_test) was used to evaluate its performance. By writing the code *y_test* and *y_train*, results can be displayed.

Creating and Training the Model

Following the established procedure, I implemented the TF-IDF Vectorizer, which converts raw review texts into a numerical format (specifically, TF-IDF vectors) that the machine learning model can process. Additionally, I utilized LinearSVC for classification tasks. It works by identifying the optimal boundary that separates data points of different classes. The machine learning Pipeline first transforms the text data into TF-IDF vectors and then fits (trains) the LinearSVC model on the training data. The implementation code is as follows:
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

```
text_clf = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('clf', LinearSVC())
])

text_clf.fit(X_train, y_train)
```

Evaluating the Model

The code used for this part is:

```
predictions = text_clf.predict(X_test)
from sklearn.metrics import confusion_matrix, classification_report
print(classification_report(y_test, predictions))
```

The model generated sentiment predictions for the test data (X_test). These predictions were then compared against the actual labels (y_test). As a result, a classification report was generated in the table format (see Table 1).

Table 1. The classification_report function's output

	precision	recall	f1-score	support
negative	0.65	0.57	0.61	306
neutral	0.60	0.64	0.62	423
positive	0.69	0.71	0.70	332
accuracy			0.64	1061
macro avg	0.65	0.64	0.64	1061
weighted avg	0.64	0.64	0.64	1061

The model's performance was evaluated using four key metrics. Precision reflects the accuracy of the predictions, while Recall shows how many actual cases were captured. The F1-score provides a balance between these two, and Support indicates the sample size for each sentiment category (Scikit-learn 2025). The explanation of the evaluation metrics was partially supported by AI-assisted tools (Google Gemini), which were used for clarification purposes.

The model achieved an overall accuracy of 64%. The best performance was observed for the positive class (F1 = 0.70), while neutral sentiment was more challenging to classify.

Visualizing the Confusion Matrix

The command used initially:

```
cm = confusion_matrix(y_test, predictions) // compares the actual correct answers (y_test) with those given by the model (predictions)
cm // creates a two-dimensional array (matrix) where the number of successful predictions and errors for each class is counted
```

Got the result:

```
array([[175, 107,  24],
```

[73, 269, 81],
[21, 74, 237]])

I visualized the classification results using a graphical table known as a Confusion Matrix. This was implemented with the following code:
from sklearn.metrics import ConfusionMatrixDisplay

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=text_clf.classes_)
disp.plot()

The results are shown in Figure 1.

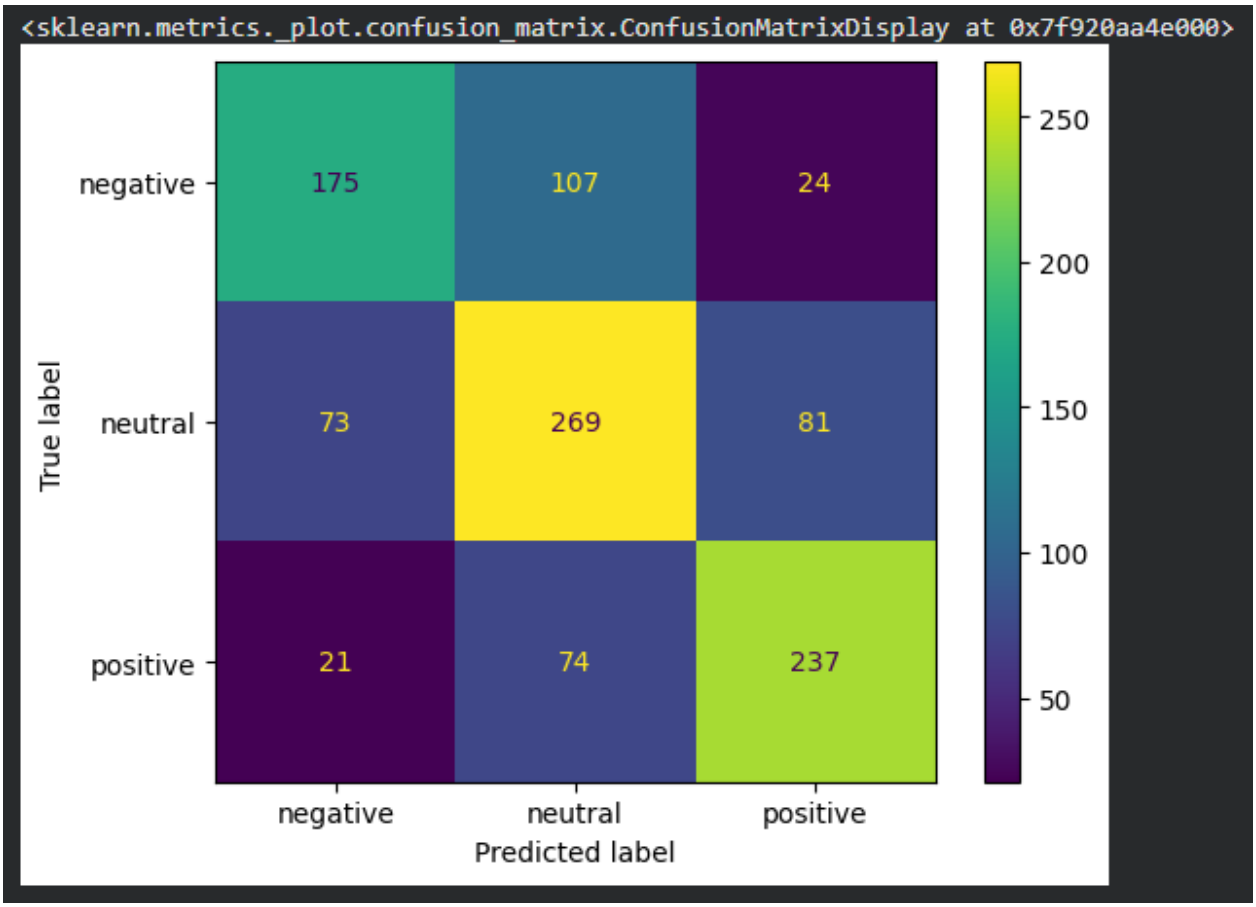

Figure 1. Confusion matrix for sentiment analysis classification

Based on the confusion matrix (Figure 1), the model most accurately identified neutral sentiments (269 correct predictions), followed by positive (237) and negative (175). The negative class was the most challenging, with 107 instances misclassified as neutral.

Making Predictions with the Model

I tested the trained model with several sample sentences, which are presented in Table 2.

Table 2. Manual testing of the model with sample text inputs

№	Input	Output
1	This is wery bad! i hate this product. do not recommend	negative
2	It looks fine	neutral
3	Something like 69.	negative
4	I am so happy to share this with you…	positive
5	I think this might end badly.	neutral

The manual testing results in Table 2 demonstrate the model's practical application. The model correctly identified clear emotional cues, such as "hate" (negative) and "happy" (positive). The model showed limitations with ambiguous or complex phrasing. For instance, in example №5, the phrase "end badly" was classified as neutral, likely because the model did not find strong enough "negative" weights in its training for that specific word combination. Example №3 shows that even neutral or obscure text like "Something like 69" can be pushed toward a class (negative) based on statistical patterns in the training data, even if the human meaning is unclear.

Conclusions

This practical assignment on sentiment analysis provided a comprehensive overview of NLP workflow in a cloud environment. By following the systematic approach of data preprocessing, model training with LinearSVC, and detailed evaluation, I gained hands-on experience in handling real-world textual data.

The analysis showed that while the model achieved a satisfactory overall accuracy of 64%, its effectiveness varies significantly across different sentiment categories. I learned that effective data cleaning, such as removing nearly 1 300 missing values, is a critical first step in ensuring the reliability of machine learning outcomes. Manual testing revealed that the model excels at identifying explicit emotional cues but struggles with subtle context and ambiguous phrasing, such as "end badly" being classified as neutral.

I now better understand how statistical patterns in training data influence predictions and the importance of using diverse metrics, like the confusion matrix, to critically evaluate a model's strengths and weaknesses. Future improvements could include hyperparameter tuning, dataset balancing, or experimenting with more advanced NLP models.

Sources

Pajula, M. 2024. Natural Language Processing (NLP) example. Google Colaboratory. Updated 2026. Accessed 11.2.2026 https://colab.research.google.com/drive/1cabxtPytrnkUNy2fGreDUvh7_R7Z4BZS?usp=sharing.

Scikit-learn. 2025. classification_report. Scikit-learn documentation. Accessed 11.2.2026 https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html.

Shrivastava, A. 2021. Sentiment Analysis Dataset. Kaggle. Accessed 11.2.2026 https://www.kaggle.com/datasets/abhi8923shriv/sentiment-analysis-dataset.

Maksym Kotlubaiev